R MOHNISH

192411112

INTERNET PROGRAMMMING

CO1 – A2 CASE STUDY

Question 1: HTTP Response Analysis

Scenario Context:
A web server returns the following HTTP response header:

http

HTTP/1.1 200 OK

Date: Mon, 27 Jul 2009 12:28:53 GMT

Server: Apache/2.2.14 (Win32)

Content-Length: -50

Content-Type: text/html

1. Incorrect or Invalid Fields Identification

- The **Content-Length** field value is explicitly invalid. In accordance with **RFC 7230**, the value must be a non-negative decimal integer representing the exact octet count of the message body. A value of -50 violates the HTTP/1.1 protocol specification.

2. Client-Side Impact Analysis

- **Data Truncation & Parsing Errors:** The client (browser) will fail to properly parse the response payload size.
- **Connection Hangs or Premature Termination:** The browser may interpret the negative value as an immediate end of data or become locked waiting infinitely for a valid transmission length, throwing a protocol violation error (e.g., ERR_INVALID_HTTP_RESPONSE).

- **Broken Keep-Alive Pipelines:** In persistent connections, an invalid payload size corrupts the stream delimiter, rendering the socket unusable for subsequent pipeline requests.

3. Protocol Conformance Determination

- The response **fails protocol conformance definitions**. While the status line, Date, Server, and Content-Type conform to proper standards, the transmission framing error directly introduces non-conformance.

4. Suggested Technical Corrections

- **Dynamic Content-Length Calculation:** The server-side configuration or backend framework must compute the precise positive length of the body layout string.
- **Alternative Chunked Transfer Encoding:** If the content length cannot be pre-computed before sending headers, the server should replace Content-Length with Transfer-Encoding: chunked.

Corrected HTTP Response Headers:

http

HTTP/1.1 200 OK

Date: Mon, 27 Jul 2009 12:28:53 GMT

Server: Apache/2.2.14 (Win32)

Content-Length: 1024

Content-Type: text/html; charset=UTF-8

Connection: keep-alive

Question 2: DOM Structure Analysis

Scenario Context:
Consider the following DOM layout structure:

html

<html>

```
  <head>
    <title>Dashboard</title>
  </head>
  <body>
    <h1>Welcome</h1>
    <p>Get started.</p>
  </div>
</html>
```

Use code with caution.

1. Mismatched or Misplaced Elements Identification

- The opening structural tag **<body>** is incorrectly paired with a closing ** ** block element. The required matching closing tag </body> is entirely omitted from the tree hierarchy.

2. Browser Interpretation & Parsing Mechanics

- Modern web browsers use the standard **HTML5 parsing algorithm** (which includes robust error-correction rules). Encountering a stray without an active matching container causes the parser to ignore the unmatched closing delimiter and continue executing structural generation.

3. Rendering & Layout Impact

- **DOM Tree Manipulation:** The parser automatically injects an implicit </body> token right before the final </html> marker to normalize the document.
- **Layout and Style Breaking:** If scripts or CSS frameworks rely on strict parent-child hierarchies (body > elements), styling rules can break down. Elements rendered afterward may shift visually, leading to unexpected layout bugs.

4. Corrected DOM Layout Structural Code Block

html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Dashboard</title>
</head>
<body>
    <h1>Welcome</h1>
    <p>Get started.</p>
</body>
</html>
```

Question 3: GET vs. POST in Login System

Scenario Context:
A login system evaluates transmission methodologies based on the following architectural behaviors:

- **GET:** /login?username=admin&pass=1234 | Visible in URL | Security Level: Low
- **POST:** /login | Hidden in HTTP body payload | Security Level: Medium

1. Data Transmission Differences

- **GET Method:** Appends name-value pairs directly to the request URI string following a ? delimiter. Data capacity is limited by browser URL length limits (typically around 2,048 characters).
- **POST Method:** Packages variables entirely inside the entity-body of the HTTP request message, removing size restrictions and keeping formatting parameters separated from routing definitions.

2. Method Suitability Analysis

- **The POST method is exclusively appropriate** for user authentication protocols. By moving identification parameters away from the URL bar, credential exposure risk across browser footprints is significantly mitigated.

3. Security Risks of GET for Authentication

- **Browser History Exposure:** Plaintext passwords remain logged permanently in local history tabs.
- **Server-Side Log Leakage:** Query parameters are automatically saved in server-side access logs (e.g., Apache/Nginx access.log), exposing administrative configurations to log-reading exploits.
- **Referer Header Leakage:** Navigating away from the landing zone passes the complete credentialed URL down inside the Referer headers of third-party assets.

4. Secure Implementation Implementation Framework

html

```
<!-- Secure Authentication Interface using HTTPS and POST -->
<form action="https://example.com" method="POST">
  <div>
    <label for="uid">Username:</label>
    <input type="text" id="uid" name="username" required>
  </div>
  <div>
    <label for="pwd">Password:</label>
    <input type="password" id="pwd" name="password" required>
  </div>
  <button type="submit">Authenticate</button>
</form>
```

Question 4: HTML Code Inspection

Scenario Context:
Analyze the following HTML snippet:

html

```
<!DOCTYPE html>
<html>
<head>
<title>Responsive Layout</title>
</head>
<body>
<div class="main">
<p>This is a paragraph
<img src="image.jpg">
</div>
</html>
```

1. Missing Closing Tags Identification

- The paragraph tag ** ** lacks its matching closing ** ** tag.
- The foundational structural block **<body>** is missing its closing **</body>** tag before the document end marker.

2. Image Tag Analysis

- The **<img>** tag is a native void element, meaning it does not need a closing tag. However, it violates fundamental web standards by lacking the mandatory **alt** (alternate text) attribute, which breaks screen-reader accessibility (WCAG compliance).

3. Browser Handling & Processing Behavior

- The browser's internal engine fixes the broken layout dynamically. When the parser encounters the block-level closing tag , it automatically closes the active paragraph element inline. It then appends the missing

</body> tag implicitly right before the </html> declaration to patch the document structure.

4. Standard-Compliant Corrected Code Block

```
html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Responsive Layout</title>
</head>
<body>
    <div class="main">
        <p>This is a paragraph</p>
        <img src="image.jpg" alt="Description of the responsive layout asset">
    </div>
</body>
</html>
```

Question 5: Browser-Server Communication Flow

Scenario Context:
The baseline lifecycle of a standard client web request follows this sequential roadmap:

1. User enters URL \(\rightarrow \) 2. DNS Lookup \(\rightarrow \) 3. HTTP Request Sent \(\rightarrow \) 4. Server Processes Request \(\rightarrow \) 5. Response Received \(\rightarrow \) 6. Page Rendering.

1. Execution Steps Detailed Overview

- **1. URL Entry:** The user inputs a web address (e.g., https://example.com), prompting the browser to parse the target protocol, hostname, and port configuration.
- **2. DNS Lookup:** The browser queries Domain Name System (DNS) servers to translate the alphanumeric hostname into an target routing IP address.
- **3. HTTP Request:** The browser initializes a TCP handshake (and TLS negotiation over HTTPS) and issues a formatted HTTP request payload to the destination server.
- **4. Server Processing:** The web server receives the payload, processes application logic, runs database operations, and generates the resulting output stream.
- **5. Response Receipt:** The server sends back the calculated data packets along with an appropriate HTTP status code (e.g., 200 OK).
- **6. Browser Rendering:** The engine parses the HTML content, builds the DOM and CSSOM trees, executes embedded javascript dependencies, and paints the final layout on screen.

2. Common Network Latency Bottlenecks

- **DNS Resolution Delays:** Uncached addresses require traversing complex, multi-tiered root name lookups.
- **Time to First Byte (TTFB):** Inefficient server-side script runtimes, slow database query execution, or resource constraints introduce major performance bottlenecks at Step 4.

3. Strategic Role of Cache Interception

- Local asset caching cuts out external round-trips entirely. If a valid copy of an asset lives locally in the browser cache, Step 2 and Step 3 are bypassed completely, serving content instantly and reducing server network strain.

4. JavaScript Implementation Example: Optimizing Flow using Async Service Workers

javascript

```
// Registering a Performance-Optimized Service Worker Cache Layer
const CACHE_NAME = 'v1_site_cache';
const ASSETS_TO_CACHE = ['/', '/styles.css', '/app.js'];

self.addEventListener('install', event => {
    event.waitUntil(
```